



Universidad
La Salle[®]
Nezahualcóyotl

ANDROID STUDIO

Evelyn Rojas Marin

11/08/26



1. CICLO DE VIDA DE LAS APLICACIONES / ACTIVITIES EN ANDROID

- **onCreate():** Se llama cuando se crea la Activity por primera vez. Aquí se hace la inicialización (setContentView, ViewModels, etc.). Recibe un Bundle con el estado guardado.
- **onStart():** La Activity se vuelve visible al usuario.
- **onResume():** La Activity está en primer plano e interactiva con el usuario. Es el estado "Resumed".
- **onPause():** La Activity pierde el foco (otra Activity se pone encima, multi-ventana, llamada, etc.). Aquí se deben pausar animaciones, liberar recursos compartidos, guardar datos ligeros.
- **onStop():** La Activity ya no es visible.
- **onDestroy():** Se llama antes de que la Activity sea destruida (por finish() o porque el sistema la mata).
- **onRestart():** Se llama cuando se vuelve a una Activity que estaba detenida.

Estados principales:

Created → Started → Resumed → Paused → Stopped → Destroyed.





2. PARA QUÉ SIRVE CADA UNA DE LAS CARPETAS DE CONFIGURACIÓN / ESTRUCTURA DEL PROYECTO

Carpeta / Archivo	Función
.gradle/	Caché de Gradle (no tocar)
.idea/	Metadatos de Android Studio (no tocar)
build.gradle(.kts) (raíz)	Configuración común de plugins
settings.gradle(.kts)	Inicialización del proyecto, módulos incluidos, repositorios
gradle.properties	Propiedades de Gradle (memoria, paralelismo, etc.)
gradlew / gradlew.bat	Wrapper de Gradle
local.properties	Rutas locales (SDK). No subir a control de versiones
gradle/libs.versions.toml	Version Catalog (versiones de dependencias)
app/	Módulo principal de la aplicación
app/build.gradle(.kts)	Configuración específica del módulo (plugins, dependencias, SDK)
app/src/main/	Código y recursos principales
app/src/main/java/ o kotlin/	Código fuente
app/src/main/res/	Recursos (layouts, drawables, values, etc.)
app/src/main/AndroidManifest.xml	Declaración de componentes, permisos, etc.
app/src/androidTest/	Tests instrumentados (en dispositivo/emulador)
app/src/test/	Tests unitarios (JVM local)
app/build/	Salidas de compilación (generado)
proguard-rules.pro	Reglas de R8/ProGuard



3. CUÁL ES EL COMPILADOR DEL ENTORNO

Sistema de build:

- Gradle + Android Gradle Plugin (AGP).

Runtime de ejecución en el dispositivo:

ART (Android Runtime), que ejecuta bytecode DEX.

Herramientas de build:

SDK Build Tools, R8 (para shrinking/obfuscation).

Lenguajes:

- Kotlin (recomendado y oficialmente preferido).
- Java (soportado).

Compilador de Kotlin:

El compilador de Kotlin integrado (en versiones recientes de AGP ya viene built-in).

Compilador de Java:

El JDK configurado (normalmente 17 o superior).

4. METODOLOGÍAS DE DESARROLLO DE APLICACIONES MÓVILES



La documentación oficial de Android no impone una metodología única, pero promueve prácticas modernas:

- **Arquitectura recomendada:**

capas separadas (UI, Domain, Data), ViewModel, Repository, flujos unidireccionales, Jetpack components.

- **Prácticas ágiles:**

desarrollo iterativo, testing continuo, modularización.

- **En la industria se usan principalmente:**

Agile / Scrum, Kanban, Lean, y enfoques híbridos. Android recomienda arquitectura limpia y testing (unitarios + instrumentados + UI tests).

5. METODOLOGÍAS / PRINCIPIOS DE DISEÑO DE APPS MÓVILES

- Material Design 3 (Material You).
- Diseño adaptativo (adaptive layouts) para teléfonos, tablets, foldables, ChromeOS, etc.
- Principios de UX: usabilidad, accesibilidad, rendimiento, privacidad y seguridad
- Guías específicas para Wear OS, TV, XR.
- Planificación de arquitectura robusta antes de codificar.
- Recursos: kits de Figma oficiales, temas y estilos de Android.



6. PARA QUÉ SIRVE CADA LÍNEA DEL CÓDIGO PRINCIPAL DE UNA ACTIVITY



```
class MainActivity : ComponentActivity() {
    override fun onCreate(savedInstanceState: Bundle?) {
        super.onCreate(savedInstanceState)
        enableEdgeToEdge()
        setContent {
            MyApplicationTheme {
                Scaffold(modifier = Modifier.fillMaxSize()) { innerPadding ->
                    Greeting(
                        name = "Android",
                        modifier = Modifier.padding(innerPadding)
                    )
                }
            }
        }
    }
}
```

Imports:

traen las clases y funciones que vas a usar.

- Bundle → para guardar/restaurar el estado de la Activity.
- ComponentActivity → la clase base moderna (recomendada) para Activities con Compose.
- setContent → función que permite poner UI hecha con Compose.
- enableEdgeToEdge → hace que la app ocupe toda la pantalla (debajo de la barra de estado y navegación).
- Las de compose.* → componentes y modificadores de la interfaz.
- MyApplicationTheme → el tema personalizado de tu app (colores, tipografía, etc.).

```
class MainActivity : ComponentActivity() {
```

Declara la clase MainActivity que hereda de ComponentActivity. Esta es la pantalla principal de la aplicación.

```
    override fun onCreate(savedInstanceState: Bundle?) {
```

Sobrescribe el método onCreate.

Es el primer método del ciclo de vida que se ejecuta cuando se crea la Activity. savedInstanceState contiene el estado anterior (si la Activity se recreó, por ejemplo al girar la pantalla).

```
        super.onCreate(savedInstanceState)
```

Llama al onCreate de la clase padre (ComponentActivity). Siempre debe hacerse primero. Si no lo pones, la Activity no se inicializa correctamente.

enableEdgeToEdge()

Activa el modo Edge-to-Edge. La app se dibuja debajo de la barra de estado y la barra de navegación (pantalla completa moderna).

setContent {

Establece el contenido de la pantalla usando Jetpack Compose. Todo lo que esté dentro de estas llaves {} es la UI declarativa.

MyApplicationTheme {

Aquí se definen los colores, tipografías y formas que usará toda la UI.

Scaffold(modifier = Modifier.fillMaxSize()) { innerPadding ->

- Scaffold es un componente de Material Design 3 que proporciona la estructura básica de una pantalla (puedes poner TopAppBar, BottomBar, FloatingActionButton, etc.).
- Modifier.fillMaxSize() → hace que ocupe todo el tamaño disponible de la pantalla.
- { innerPadding -> ... } → es una lambda que recibe el padding interno que el Scaffold calcula (para que el contenido no quede debajo de la barra de estado o de navegación).

Greeting(

name = "Android",

modifier = Modifier.padding(innerPadding))

- Llama a la función composable Greeting.
- Le pasa el texto "Android" y le aplica el padding que calculó el Scaffold para que el texto no quede oculto detrás de las barras del sistema.

Resumen del flujo

1. **Se crea la Activity → onCreate**
2. **Se habilita pantalla completa → enableEdgeToEdge**
3. **Se pone el contenido Compose → setContent**
4. **Se aplica el tema → MyApplicationTheme**
5. **Se usa un Scaffold como estructura**
6. **Dentro se muestra el texto "Hello Android!" con el padding correcto**

7. COMPONENTES / CARACTERÍSTICAS PRINCIPALES DE ANDROID STUDIO



Componente	Explicación breve
Toolbar	Barra superior con botones de acciones rápidas (Run, Debug, Sync, etc.).
Navigation Bar	Muestra la ruta del archivo actual y permite navegar rápidamente por el proyecto.
Editor Window	Zona principal donde escribes y editas el código.
Tool Window Bar	Barra lateral con botones para abrir/cerrar las ventanas de herramientas.
Status Bar	Muestra información del proyecto, errores, progreso de builds y estado del IDE.
Project	Muestra la estructura de archivos y carpetas del proyecto.
Structure	Vista rápida de la estructura de clases, métodos y variables del archivo actual.
Favorites	Permite marcar archivos o carpetas favoritas para acceso rápido.
Build	Muestra el progreso y resultados de la compilación del proyecto.
Run	Ventana de ejecución de la aplicación (logs de salida).
Debug	Herramientas para depurar código (breakpoints, variables, step by step).
Logcat	Muestra los logs del sistema y de la aplicación en tiempo real.
Profiler	Analiza el rendimiento (CPU, memoria, red y batería) de la app.
Layout Inspector	Inspecciona la jerarquía de vistas de la UI en tiempo real.
Device Manager	Gestiona emuladores y dispositivos físicos conectados.
Emulator	Simula dispositivos Android para probar la aplicación.
Version Control	Integración con Git para control de versiones.

Terminal	Terminal integrada para ejecutar comandos.
Problems	Lista de errores, warnings y problemas detectados en el código.
TODO	Muestra todos los comentarios TODO del proyecto.
Resource Manager	Gestiona y previsualiza recursos (imágenes, strings, layouts, etc.).
App Inspection	Inspecciona bases de datos, red y tareas en segundo plano de la app.
Database Inspector	Visualiza y consulta las bases de datos de la aplicación en tiempo real.
Network Inspector	Analiza las peticiones de red que realiza la app.
Background Task Inspector	Supervisa trabajos en segundo plano (WorkManager, etc.).
Layout Editor	Editor visual para diseñar interfaces con Views (XML).
Compose Preview	Vista previa interactiva de componentes Jetpack Compose.
Animation Preview	Permite previsualizar y depurar animaciones de Compose.
Live Edit	Actualiza la UI de Compose en tiempo real sin recompilar completamente.
Lint	Analiza el código en busca de errores, problemas de rendimiento y buenas prácticas.
Build Analyzer	Analiza el tiempo de compilación y detecta cuellos de botella.
Gradle	Sistema de construcción del proyecto (gestiona dependencias y builds).
SDK Manager	Instala y actualiza SDKs, emuladores y herramientas de Android.
AVD Manager	Crea y administra Android Virtual Devices (emuladores).
Gemini	Asistente de IA integrado para generar código, explicar y corregir errores.
Code Completion	Autocompletado inteligente de código mientras escribes.
Refactor	Herramientas para renombrar, mover y reestructurar código de forma segura.

Find Usages	Busca todas las referencias de una clase, método o variable.
Apply Changes	Aplica cambios de código a la app en ejecución sin reiniciarla completamente.
Device Streaming	Permite probar la app en dispositivos reales remotos desde Android Studio.
App Links Assistant	Ayuda a configurar y verificar App Links / Deep Links.
Translations Editor	Facilita la gestión de cadenas de texto en múltiples idiomas.
Theme Editor	Permite editar y previsualizar temas de Material Design.
Memory Profiler	Detecta fugas de memoria y analiza el uso de RAM.
CPU Profiler	Analiza el uso de procesador y métodos que consumen más tiempo.

REFERENCIAS:

- Android Developers. (n.d.). The activity lifecycle. Android Developers.
<https://developer.android.com/guide/components/activities/activity-lifecycle>
- Android Developers. (n.d.). Introduction to activities. Android Developers.
<https://developer.android.com/guide/components/activities/intro-activities>
- Android Developers. (n.d.). Tools for Compose. Android Developers.
<https://developer.android.com/develop/ui/compose/tooling>
- Android Developers. (n.d.). Guide to app architecture. Android Developers. <https://developer.android.com/topic/architecture>
- Android Developers. (n.d.). Projects overview. Android Developers. <https://developer.android.com/studio/projects>
- Android Developers. (n.d.). Android build structure. Android Developers. <https://developer.android.com/build/android-build-structure>